

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Тульский государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине
Методы и средства защиты информации
по теме
«Программная реализация алгоритмов шифрования»

Выполнил: ст. гр. 220032-11

Лаукерт К. М.

Принял: к.т.н., доцент ИПМКН

Кулешова Н. В.

Тула 2026

Лабораторная работа №1

по теме «Программная реализация алгоритмов шифрования»

Цель работы

Изучить и научиться применять различные методы симметричного шифрования.

Задание

В соответствии с вариантом 9 реализовать программно два метода симметричного шифрования, не используя криптографические библиотеки. Для каждого метода зашифровать и дешифровать исходную фразу длиной не менее 20 символов.

Реализованы шифр Гронсфельда и шифр двойной перестановки. Исходный текст: «Знание криптографии защищает данные» (35 символов).

Описание алгоритма шифра Гронсфельда

Шифр Гронсфельда является многоалфавитной подстановкой и модификацией шифра Цезаря. В качестве ключа используется последовательность цифр. Каждая цифра задаёт величину циклического сдвига соответствующей буквы русского алфавита. Если ключ короче сообщения, он повторяется циклически. При расшифровании из позиции буквы вычитается соответствующая цифра ключа. Пробелы не изменяются и не расходуют символ ключа.

В программе использован цифровой ключ 314159 и русский алфавит из 33 букв, включая букву Ё. Для каждой буквы с номером p и цифры ключа k вычисляется номер шифртекста $c = (p + k) \bmod 33$. При дешифровании используется $p = (c - k) \bmod 33$.

Описание алгоритма двойной перестановки

При двойной перестановке исходный текст записывается в прямоугольную таблицу по строкам. Сначала столбцы переставляются в соответствии с ключом столбцов, затем строки — в соответствии с ключом строк. Шифртекст получают последовательным считыванием заполненной таблицы по строкам.

В реализации использован ключ столбцов [3, 1, 4, 2, 5]. Текст размещён в таблице из 7 строк и 5 столбцов; ключ строк [7, 6, 5, 4,

3, 2, 1]. При необходимости последняя строка дополняется символом X. Для расшифрования перестановки строк и столбцов выполняются в обратном порядке, после чего удаляется добавленное дополнение по сохранённой исходной длине сообщения.

Программная реализация

Файл программы: ciphers.py. Реализация выполнена на языке Python без использования библиотек шифрования.

```
"""Лабораторная работа №1: реализация симметричных шифров без библиотек."""
```

```
from math import ceil
```

```
RUSSIAN_ALPHABET = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
```

```
def gronsfeld_encrypt(text: str, key: str) -> str:
```

```
    """Шифрует текст шифром Гронсфеля с цифровым ключом."""
```

```
    _validate_digit_key(key)
```

```
    return _gronsfeld(text, key, 1)
```

```
def gronsfeld_decrypt(text: str, key: str) -> str:
```

```
    """Расшифровывает текст, зашифрованный шифром Гронсфеля."""
```

```
    _validate_digit_key(key)
```

```
    return _gronsfeld(text, key, -1)
```

```
def _validate_digit_key(key: str) -> None:
```

```
    if not key or not key.isdigit():
```

```
        raise ValueError("Ключ Гронсфеля должен содержать хотя бы одну цифру.")
```

```
def _gronsfeld(text: str, key: str, direction: int) -> str:
```

```

result = []
key_index = 0
alphabet_size = len(RUSSIAN_ALPHABET)

for symbol in text.upper():
    if symbol in RUSSIAN_ALPHABET:
        position = RUSSIAN_ALPHABET.index(symbol)
        shift = int(key[key_index % len(key)])
        result.append(RUSSIAN_ALPHABET[(position + direction
* shift) % alphabet_size])
        key_index += 1
    else:
        result.append(symbol)
return "".join(result)

def double_transposition_encrypt(
    text: str, column_key: list[int], row_key: list[int]
) -> tuple[str, int]:
    """Шифрует текст двойной перестановкой и возвращает
    шифртекст с исходной длиной."""
    _validate_permutation(column_key)
    _validate_permutation(row_key)

    columns = len(column_key)
    rows = len(row_key)
    capacity = rows * columns
    if len(text) > capacity:
        raise ValueError("Длина текста превышает размер таблицы
для выбранных ключей.")

    padded = text.upper().ljust(capacity, "X")
    table = [list(padded[index:index + columns]) for index in
range(0, capacity, columns)]
    reordered_columns = [[row[index - 1] for index in
column_key] for row in table]
    reordered = [reordered_columns[index - 1] for index in
row_key]

```

```

    return "".join("".join(row) for row in reordered), len(text)

def double_transposition_decrypt(
    ciphertext: str, column_key: list[int], row_key: list[int],
    original_length: int
) -> str:
    """Восстанавливает исходный текст по шифртексту и
    сохранённой исходной длине."""
    _validate_permutation(column_key)
    _validate_permutation(row_key)

    columns = len(column_key)
    rows = len(row_key)
    if len(ciphertext) != rows * columns:
        raise ValueError("Длина шифртекста не соответствует
        размеру таблицы.")
    if not 0 <= original_length <= len(ciphertext):
        raise ValueError("Некорректная исходная длина текста.")

    encrypted_table = [list(ciphertext[index:index + columns])
        for index in range(0, len(ciphertext), columns)]
    after_row_restore = [None] * rows
    for current_index, original_index in enumerate(row_key):
        after_row_restore[original_index - 1] =
            encrypted_table[current_index]

    restored = []
    for row in after_row_restore:
        original_row = [None] * columns
        for current_index, original_index in
            enumerate(column_key):
            original_row[original_index - 1] =
                row[current_index]
        restored.extend(original_row)
    return "".join(restored)[:original_length]

```

```

def _validate_permutation(key: list[int]) -> None:
    if not key or sorted(key) != list(range(1, len(key) + 1)):
        raise ValueError("Ключ перестановки должен содержать
числа от 1 до длины ключа без повторов.")

def main() -> None:
    plaintext = "Знание криптографии защищает данные"
    gronsfeld_key = "314159"
    column_key = [3, 1, 4, 2, 5]
    row_count = ceil(len(plaintext) / len(column_key))
    row_key = list(range(row_count, 0, -1))

    print("Лабораторная работа №1 – симметричные шифры")
    print("=" * 60)
    print(f"Исходный текст ({len(plaintext)} символов):
{plaintext}")
    print()

    gronsfeld_ciphertext = gronsfeld_encrypt(plaintext,
gronsfeld_key)
    print("1. Шифр Гронсфельда")
    print(f"Ключ: {gronsfeld_key}")
    print(f"Шифртекст: {gronsfeld_ciphertext}")
    print(f"Расшифрованный текст:
{gronsfeld_decrypt(gronsfeld_ciphertext, gronsfeld_key)}")
    print()

    transposition_ciphertext, original_length =
double_transposition_encrypt(
    plaintext, column_key, row_key
)
    restored_text = double_transposition_decrypt(
    transposition_ciphertext, column_key, row_key,
original_length
)
    print("2. Шифр двойной перестановки")
    print(f"Ключ столбцов: {column_key}")

```

```

print(f"Ключ строк: {row_key}")
print(f"Шифртекст: {transposition_ciphertext}")
print(f"Расшифрованный текст: {restored_text}")
print()

if restored_text == plaintext.upper() and
gronsfeld_decrypt(gronsfeld_ciphertext, gronsfeld_key) ==
plaintext.upper():
    print("Проверка пройдена: оба алгоритма корректно
восстанавливают исходный текст.")
else:
    raise RuntimeError("Проверка обратимости шифрования не
пройдена.")

if __name__ == "__main__":
    main()

```

Результат выполнения

```

Lab1_Result
Лабораторная работа №1 – симметричные шифры
=====
Исходный текст (35 символов): Знание криптографии защищает данные

1. Шифр Гронсфельда
Ключ: 314159
Шифртекст: КОДОНН НСМРЧЧЁСДХНС КБЭЙЮИЗУ ЗБТЦЮЁ
Расшифрованный текст: ЗНАНИЕ КРИПТОГРАФИИ ЗАЩИЩАЕТ ДАННЫЕ

2. Шифр двойной перестановки
Ключ столбцов: [3, 1, 4, 2, 5]
Ключ строк: [7, 6, 5, 4, 3, 2, 1]
Шифртекст: НАЫНТА ЕДЦЗИАИИФ ОПГТРКЕР ИАЗНИИ
Расшифрованный текст: ЗНАНИЕ КРИПТОГРАФИИ ЗАЩИЩАЕТ ДАННЫЕ

Проверка пройдена: оба алгоритма корректно восстанавливают исходный текст.
C:\Users\PC\Documents\ТулГУ\6_семестр\Методы_и_средства_защиты>

```

Рисунок 1 — фактический запуск программы шифрования и расшифрования

Программа зашифровала исходную фразу обоими алгоритмами и успешно восстановила её при расшифровании. Для шифра Гронсфельда получен шифртекст «КОДОНН НСМРЧЧЁСДХНС КБЭЙЮИЗУ ЗБТЦЮЁ». Для двойной перестановки получен

шифртекст «НАЫНТА ЕДЦЗИАЦИАИФ ОПГТРКЕР
ИАЗННИ».

Вывод

В ходе работы были реализованы два метода симметричного шифрования: шифр Гронсфельда и шифр двойной перестановки. Для каждого алгоритма выполнено шифрование и обратное преобразование фразы длиной более 20 символов. Проверка показала, что после расшифрования в обоих случаях получается исходный текст. Шифр Гронсфельда изменяет символы по ключу, а двойная перестановка меняет их порядок в таблице.